Software Design Document (SDD)

Module: utils - Utility Library Module

Version: 1.0

Authors: Steven McClellan, Vania Leal, Leonardo Garcia, Melanie Picen

Item	Description
Source files	utils.c / utils.h
Main purpose	Provide lightweight string formatting, integer/ASCII conversion, memory copy, and memory reverse helpers for embedded modules.
Main public functions	utils_snprintf(), utils_vsnprintf(), utils_itoa(), utils_atoi(), utils_memCpy(), utils_memReverse()

1. Introduction

1.1 Purpose

This document describes the design and architecture of the utils module. The module provides lightweight utility functions for embedded C applications, including formatted string generation, integer-to-ASCII conversion, ASCII-to-integer conversion, memory copying, and byte-order reversal. It avoids dynamic memory allocation and provides small helper functions that can be used by higher-level modules such as serial formatting and debugging code.

1.2 Scope

The utils module is a software-only support layer. It does not access hardware registers and does not depend on MCU peripherals. It operates only on caller-provided buffers and values. The module implements a reduced snprintf-style formatter supporting %s, %d, %u, %x, %c, and %% specifiers, as well as conversion and memory helpers intended for bare-metal projects where the full C standard library may not be desired.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ASCII	Character encoding used to represent printable characters and digits.
Buffer	Caller-provided memory region used to store strings or copied bytes.
Base	Numerical base used for conversion: binary, octal, decimal, or hexadecimal.
va_list	C variable argument list used by utils_vsnprintf().
SDD	Software Design Document.
API	Application Programming Interface.

1.4 Requirements Traceability

Req. ID	Function	Requirement Statement
FR-1	utils_snprintf()	The system shall format a string using variable arguments and store the result in a destination buffer.
FR-2	utils_vsnprintf()	The system shall format a string using an existing va_list and store the result in a destination buffer.
FR-3	utils_itoa()	The system shall convert a signed or unsigned integer into a null-terminated ASCII string in base 2, 8, 10, or 16.
FR-4	utils_atoi()	The system shall convert an ASCII string into an integer value in base 2, 8, 10, or 16.
FR-5	utils_memCpy()	The system shall copy a block of bytes from source to destination.
FR-6	utils_memReverse()	The system shall reverse the order of bytes in a caller-provided memory region.
FR-7	utils_printString() / utils_printInt()	The system shall use internal helper functions to copy strings and formatted integer text into destination buffers.

Non-functional requirements: no dynamic memory allocation; no hardware dependencies; small footprint suitable for bare-metal embedded systems; caller owns buffer sizing; functions use simple loops and fixed-size temporary storage where applicable.

2. System Architecture

2.1 High-Level Design

The utils module sits below application and communication modules as a reusable support library. Higher-level modules call its public API when they need formatted output, number conversion, or simple memory manipulation. The module depends only on standard C type definitions and variable argument support.

Layer	Files / Components	Description
Application / Higher-Level Modules	main.c, serial.c, debug modules	Call utility functions to build strings, convert numbers, or manipulate small buffers.
Utilities Layer	utils.c / utils.h	Implements formatting, conversion, copy, and reverse operations.
C Support Layer	stdint.h, stddef.h, stdarg.h	Provides fixed-width types, size_t, and va_list support.

2.2 Class Diagram

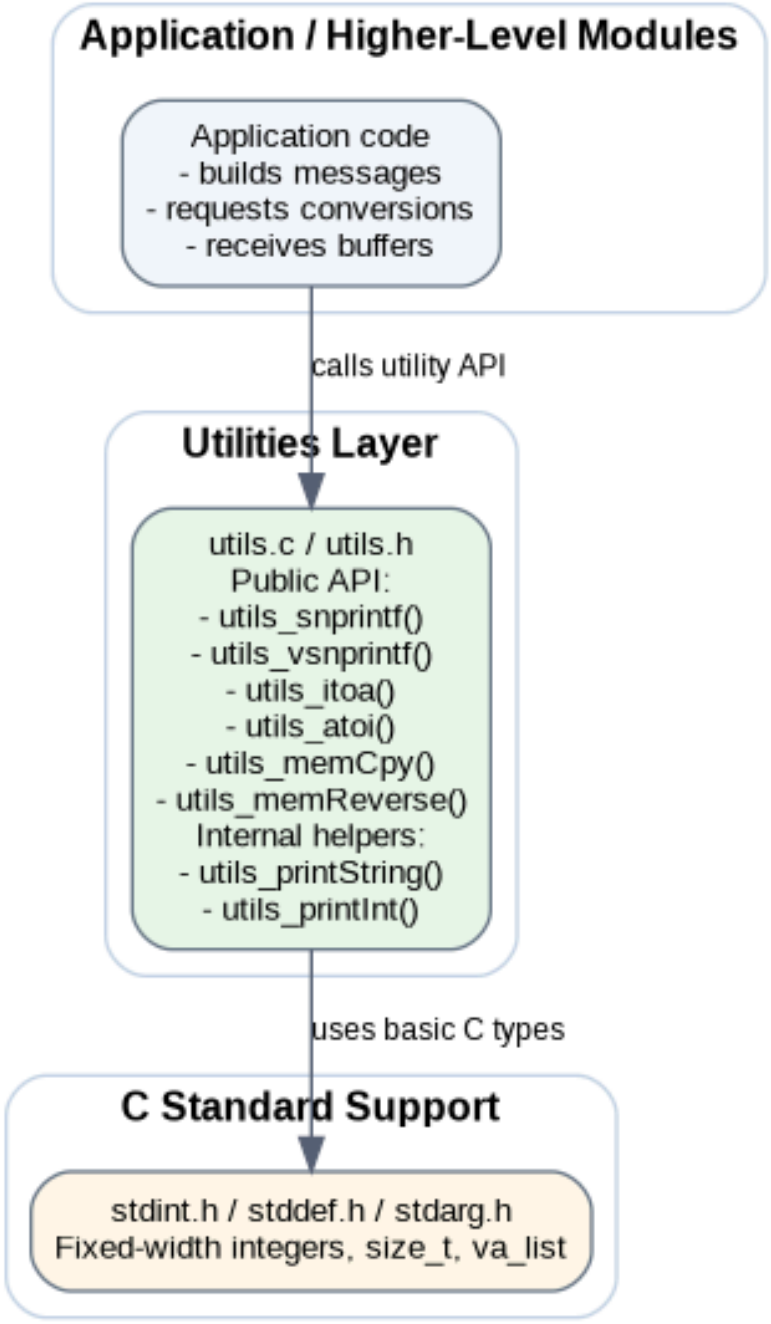

Figure 1. Class Diagram - utils module.

2.3 Module Interfaces

Interface	Description
Public API	utils_snprintf(), utils_vsnprintf(), utils_itoa(), utils_atoi(), utils_memCpy(), utils_memReverse() declared in utils.h.
Private helpers	utils_printString() and utils_printInt(), used internally by the formatting functions.
External dependencies	stdint.h, stddef.h, stdarg.h, and project-level utils.h definitions for base and sign macros.

3. Detailed Design

3.1 Module Behavior

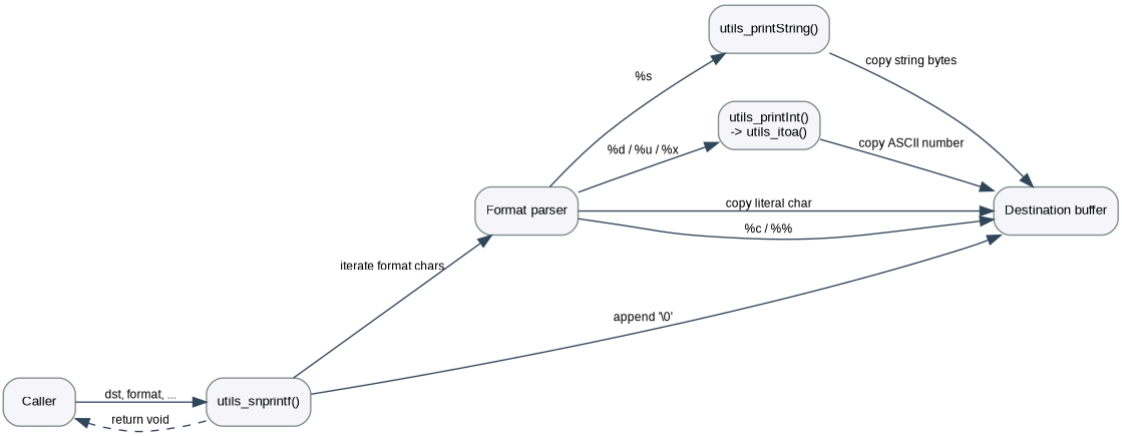

Figure 2. Sequence Diagram - utils_snprintf formatting flow.

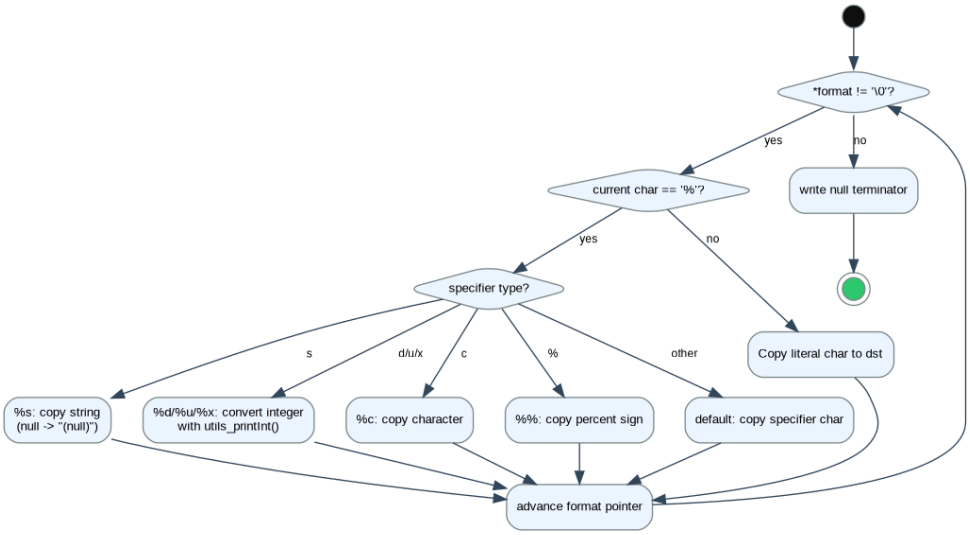

Figure 3. Activity Diagram - utils_snprintf / utils_vsnprintf parser.

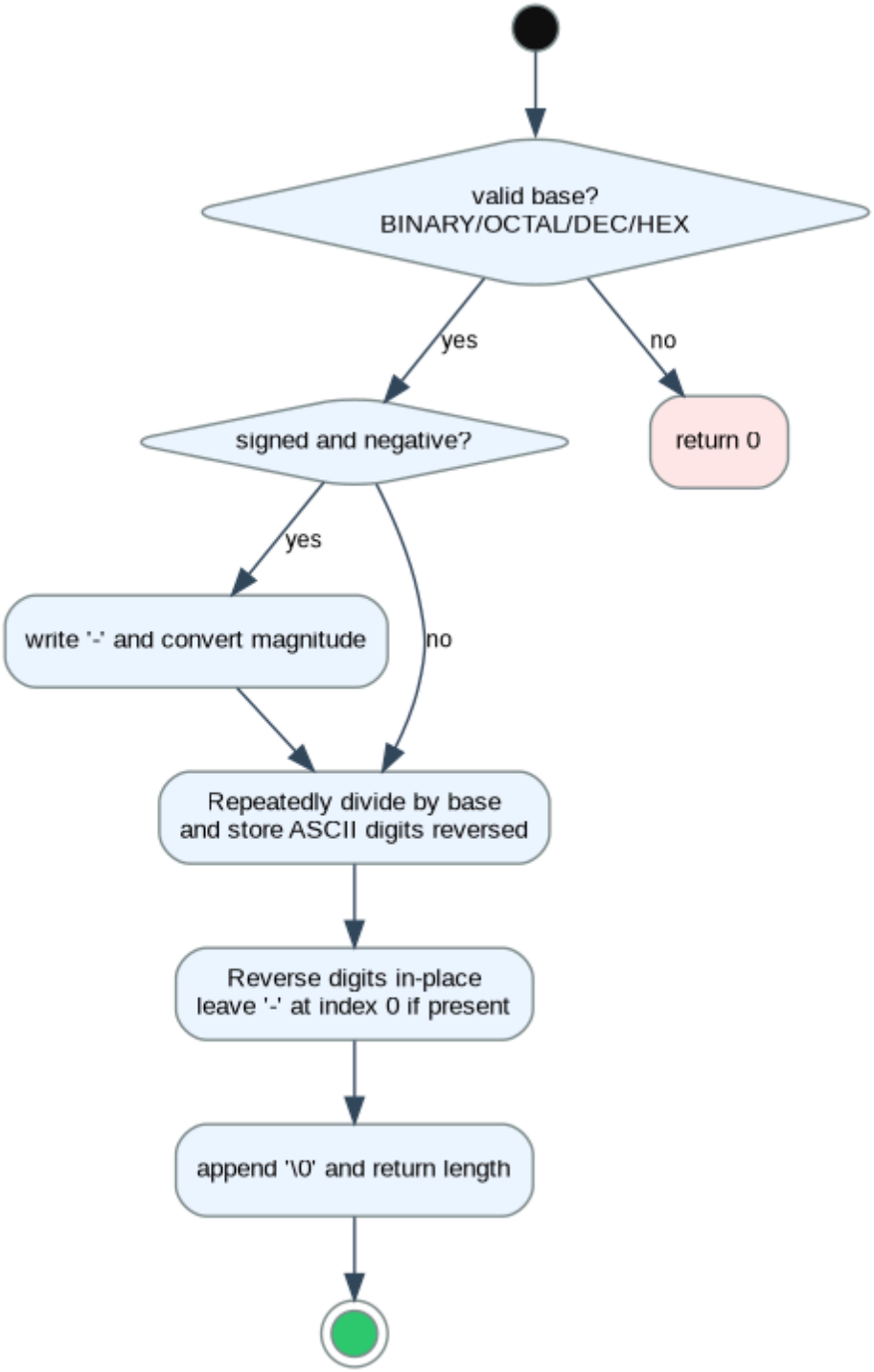

Figure 4. Activity Diagram - utils_itoa conversion flow.

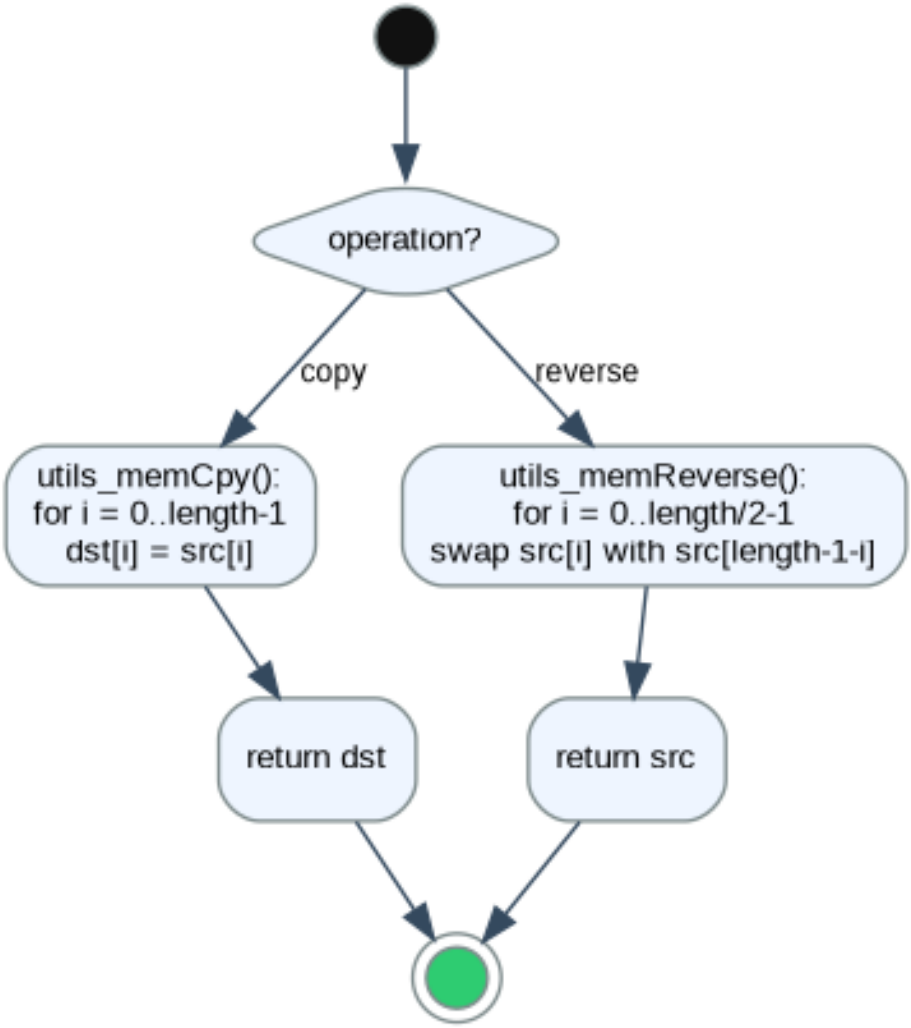

Figure 5. Activity Diagram - utils_memCpy and utils_memReverse.

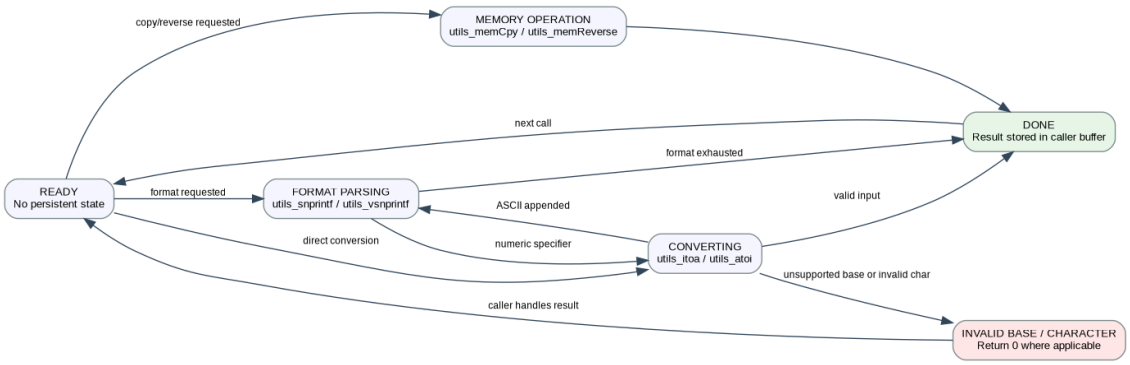

Figure 6. State Diagram - utils module lifecycle.

3.2 utils Module

The module has no persistent state. Each public function completes its operation using the parameters supplied by the caller. Formatting functions walk through the format string one character at a time. Literal characters are copied directly to the destination buffer, while conversion specifiers trigger either string copying or integer conversion. At the end of formatting, the destination buffer is terminated with a null character.

3.3 Formatting Strategy

utils_snprintf() creates a va_list using va_start(), then parses the format string. utils_vsnprintf() receives an already-created va_list and uses the same parser logic. Supported specifiers are %s for strings, %d for signed decimal, %u for unsigned decimal, %x for unsigned hexadecimal, %c for a single character, and %% for a literal percent sign. Unsupported specifiers are copied as literal characters after the percent sign is consumed.

3.4 Integer Conversion Strategy

utils_itoa() first maps the requested base macro to a numeric base value. It handles signed negative decimal values by writing the minus sign and converting the magnitude using an int64_t cast to avoid overflow on the most negative int32_t value. Digits are generated in reverse order through repeated modulo/division, then reversed in place. utils_atoi() performs the inverse operation by validating each digit against the selected base and accumulating the numeric value.

3.5 Memory Helper Strategy

utils_memCpy() copies bytes from src to dst using uint8_t pointers. It assumes the regions do not overlap. utils_memReverse() swaps bytes from the ends of a buffer toward the center until the selected length is reversed. Both functions return the pointer to the modified destination/source buffer, matching common C utility conventions.

4. Application Programming Interfaces

4.1 Data Types and Macros

Macro / Type	Value / Role	Notes
ASCII_SIGN	45u	ASCII code for the minus sign. Defined but not required by the current itoa implementation because the literal '-' is written directly.
ASCII_NUM_OFFSET	'0'	Offset used to convert numeric digits 0-9 to ASCII.
ASCII_CHAR_OFFSET	'A'	Offset used to convert hexadecimal digits 10-15 to A-F.
SIGNED / UNSIGNED	Defined in utils.h	Selects signed or unsigned conversion behavior.
BINARY / OCTAL / DECIMAL / HEX	Defined in utils.h	Selects base 2, 8, 10, or 16 for conversion functions.
uint8_t, uint32_t, int32_t, size_t	Standard integer/size types	Used for buffer data, counters, return lengths, and conversion values.

4.2 Error Codes

The utils module does not define a status-code enum. Error or unsupported cases are handled through return values where applicable. Formatting functions return void, so the caller is responsible for providing valid pointers and sufficient buffer space.

Function	Error / Limitation Behavior
utils_itoa()	Returns 0 if the requested base is unsupported.
utils_atoi()	Returns 0 if the base is unsupported, if a character is invalid, or if a digit is outside the selected base. This is ambiguous with a valid numeric value of 0.
utils_snprintf() / utils_vsnprintf()	No buffer length parameter is used; caller must ensure dst has enough space.
utils_memCpy()	No NULL or overlap checks are performed.
utils_memReverse()	No NULL check is performed.

4.3 Function Definitions

utils_snprintf

Field	Definition

Function Name	utils_snprintf
Syntax	void utils_snprintf(char *dst, const char *format, ...)
Parameters (in)	dst: destination buffer; format: format string; ...: variable arguments.
Parameters (inout)	None.
Parameters (out)	None.
Return value	void
Description	Formats text into dst using supported conversion specifiers.
Constraints	Caller must provide valid pointers and enough destination buffer capacity.
Available via	utils.h

utils_vsnprintf

Field	Definition
Function Name	utils_vsnprintf
Syntax	void utils_vsnprintf(char *dst, const char *format, va_list args)
Parameters (in)	dst: destination buffer; format: format string; args: existing variable argument list.
Parameters (inout)	args is consumed as arguments are read.
Parameters (out)	None.
Return value	void
Description	Formats text into dst using an already initialized va_list.
Constraints	Caller manages va_list lifetime and buffer capacity.
Available via	utils.h

utils_itoa

Field	Definition
Function Name	utils_itoa
Syntax	uint32_t utils_itoa(int32_t data, uint8_t *ptr, uint8_t sign, uint8_t base)
Parameters (in)	data: number to convert; sign: signed/unsigned selector; base: numeric base selector.
Parameters (inout)	ptr is written with ASCII output.
Parameters (out)	ptr: null-terminated ASCII string.
Return value	uint32_t length, or 0 if base is invalid.
Description	Converts integer data into ASCII representation.
Constraints	ptr must be valid and large enough; base must be BINARY/OCTAL/DECIMAL/HEX.
Available via	utils.h

utils_atoi

Field	Definition
Function Name	utils_atoi
Syntax	int32_t utils_atoi(uint8_t *ptr, uint32_t digits, uint8_t sign, uint8_t base)
Parameters (in)	ptr: ASCII input; digits: maximum digits to parse; sign: signed/unsigned selector; base: numeric base selector.
Parameters (inout)	None.
Parameters (out)	None.
Return value	int32_t converted value, or 0 on invalid input/base.
Description	Converts ASCII digits to an integer using the selected base.
Constraints	Invalid input and valid 0 both return 0, so caller cannot distinguish them.
Available via	utils.h

utils_memCpy

Field	Definition
Function Name	utils_memCpy
Syntax	void * utils_memCpy(void *dst, void *src, size_t length)
Parameters (in)	dst: destination; src: source; length: number of bytes.
Parameters (inout)	dst memory is overwritten.
Parameters (out)	dst contains copied bytes.
Return value	void* pointer to dst.

Description	Copies length bytes from src to dst.
Constraints	No overlap or NULL checks; overlapping regions are not supported.
Available via	utils.h

utils_memReverse

Field	Definition
Function Name	utils_memReverse
Syntax	void * utils_memReverse(void *src, size_t length)
Parameters (in)	src: data buffer; length: number of bytes to reverse.
Parameters (inout)	src is modified in place.
Parameters (out)	src bytes reversed.
Return value	void* pointer to src.
Description	Reverses byte order in the provided buffer.
Constraints	No NULL check; works byte-wise, not element-wise for multi-byte objects.
Available via	utils.h

4.4 Internal Function Definitions

Function	Syntax	Description
utils_printString	static uint32_t utils_printString(char *dst, char *src)	Measures the source string length and copies its bytes into the destination buffer using utils_memCpy().
utils_printInt	static uint32_t utils_printInt(char *dst, int32_t num, uint8_t sign, uint8_t base)	Converts an integer to ASCII using utils_itoa(), then copies the resulting bytes into the destination buffer.

5. Implementation Notes and Constraints

Topic	Note
No buffer-size checking	utils_snprintf() and utils_vsnprintf() do not receive a maximum length parameter, so they can overflow the destination buffer if the caller provides insufficient space.
No NULL validation	Several functions assume pointers are valid. This is lightweight but places responsibility on the caller.
itoa temporary buffer	utils_printInt() uses a 12-byte local buffer, enough for signed 32-bit decimal including minus sign and terminator, but not necessarily for extended future widths.
atoi ambiguity	utils_atoi() returns 0 both for invalid input and for a valid numeric zero.
Memory copy overlap	utils_memCpy() assumes source and destination do not overlap. It is not a memmove replacement.
Hex output	utils_itoa() emits uppercase A-F for hexadecimal digits.

6. Future Enhancements

- Add buffer-length parameters to the formatting functions to prevent buffer overflow.
- Return explicit status codes for conversion failures instead of using 0 as an error value.
- Add NULL pointer validation for memory and formatting functions.
- Support additional printf specifiers or field widths if needed by future modules.
- Add lowercase hexadecimal output as an option.
- Add unit tests for signed conversion, invalid bases, invalid ASCII digits, memory copy, and memory reverse behavior.